Politechnika Wrocławska · Wydział Informatyki i Telekomunikacji

RAPORT – ETAP 2

Pomiar wyjściowy i pierwsza optymalizacja serwisu ByShutter

Zmniejszenie zdjęć, konwersja do WebP i zachowanie oryginałów

Temat projektu	Badanie wydajności ładowania zasobów (w szczególności zdjęć) w serwisie portfolio fotograficznego ByShutter.
Zespół	Adrian Czubaty, Jan Narożny
Zakres etapu	Pomiar stanu wyjściowego na dużych zdjęciach, wdrożenie pierwszej optymalizacji i ponowny pomiar w identycznych warunkach.
Narzędzia	Google Lighthouse (sterowany z Node.js, ograniczanie łącza i CPU), pomiary czasów pobierania programem `curl`. WebPageTest nie został użyty (p. rozdz. 2.5).
Optymalizacja	Przy wgrywaniu: zmniejszenie zdjęcia do 2048 px, konwersja do WebP (jakość 80), zachowanie oryginału oraz przycisk „View full image”.
Data	4 listopada 2026

Podsumowanie

Zmierzyliśmy serwis w stanie wyjściowym na siedmiu dużych zdjęciach (ok. 30 MB), wdrożyliśmy pierwszą optymalizację (zmniejszenie zdjęć do 2048 px, konwersja do WebP przy wgrywaniu, oryginały zachowane i dostępne przyciskiem „View full image”) i zmierzyliśmy efekt w identycznych warunkach. Strona główna pobiera teraz 3,14 MB zamiast 24,16 MB (−87 %), a LCP skrócił się z 21,8 do 3,74 s na profilu 4G i z 36,9 do 4,84 s na Cable. W albumie LCP spadł z 20,8 do 2,28 s (4G) i z 42,8 do 2,90 s (Cable), a Total Blocking Time z 1,5–3,0 s do zera. Kosztem jest jednorazowo dłuższe wgrywanie zdjęcia (ok. 2–4 s zamiast 40 ms) i o 13 % większe zużycie dysku. Strona główna wciąż przekracza próg 2,5 s dla LCP, bo sześć zdjęć w rozmiarze 2048 px dzieli łącze; wskazujemy kolejne kroki (warianty rozmiarów, `srcset`, priorytet zdjęcia tytułowego).

1 Zakres i przebieg etapu

Etap 2 realizuje plan zapowiedziany w raporcie 1: zmierzyć stan wyjściowy serwisu, wprowadzić pierwszą optymalizację zdjęć i zmierzyć efekt tak samo jak wcześniej. Przebieg był następujący:

1. uruchomienie aplikacji w stanie wyjściowym (Docker Compose: PostgreSQL, API ASP.NET Core, Caddy) wraz z klientem `ByShutterPage`,
2. wyczyszczenie bazy danych (kontenery utworzone od nowa, tabele z treścią puste),
3. wgranie dużych zdjęć przez API (siedem plików, ok. 30 MB, te same, które opisaliśmy w raporcie 1) i przygotowanie treści: profil strony głównej, cztery kafelki, album,
4. pomiar stanu wyjściowego (5 przebiegów na kombinację strony i profilu sieci, mediana),
5. wdrożenie optymalizacji nr 1 w backendzie,
6. ponowne wyczyszczenie bazy, wgranie tych samych zdjęć i ponowny pomiar w identycznej konfiguracji,
7. porównanie wyników i wnioski.

Względem raportu 1 zmieniło się jedno: WebPageTest zastąpiliśmy lokalnym Lighthouse (rozdz. 2.5).

2 Środowisko i metodyka pomiarów

2.1 Środowisko

Element	Wartość
Komputer	Apple M2 Pro, macOS 27.0.1; wszystkie składniki (przeglądarka, Docker, serwer plików) na jednej maszynie, adres `localhost`
Docker	Docker Desktop (silnik 29.4.3), 12 procesorów, ok. 8,3 GB pamięci dla maszyny wirtualnej
Kontenery	`postgres:16-alpine`, `caddy:2-alpine` (serwer plików, HTTPS na porcie 5702), API zbudowane z `mcr.microsoft.com/dotnet/sdk:10.0` (ASP.NET Core, .NET 10)
Klient	`ByShutterPage` (Vue 3.5, Vite 8, Tailwind CSS 4), wersja produkcyjna (`vite build`), serwowana przez `vite preview` na porcie 5173
Przeglądarka	Chrome for Testing 156.0.8078.80, tryb headless, nowa instancja przy każdym pomiarze
Narzędzie	Lighthouse 13.6.0 sterowany z Node.js v26.0.0 (kategoria *performance*)

Tabela 1: Środowisko pomiarowe

2.2 Dane testowe

Użyliśmy siedmiu dużych zdjęć z katalogu `site/` repozytorium, czyli tych samych plików, które opisaliśmy w raporcie 1: sześć zdjęć z telefonu (4032 × 3024 px, od 3,2 do 5,2 MB) i jedno zdjęcie z serwisu Pexels (6240 × 4160 px, 5,9 MB), łącznie ok. 30,0 MB. Pliki wgrywane są przez `POST /api/photos` pod nowymi nazwami `sample-01` do `sample-07`, żeby nie nadpisać plików śledzonych w repozytorium. Układ treści: zdjęcie 1 jest zdjęciem tytułowym, zdjęcie 2 profilowym, zdjęcia 3–6 to cztery kafelki portfolio, a album zawiera wszystkie siedem zdjęć. Skrypt `bench/seed.mjs` odmawia pracy na niepustej bazie, dzięki czemu oba pomiary startują z identycznego stanu.

2.3 Klient użyty w pomiarach

Mierzymy klienta `ByShutterPage` (Vue 3, TypeScript, Vite, Tailwind CSS 4, minimalistyczny wygląd portfolio) w wersji produkcyjnej. Zachowania ładowania istotne dla badania są takie, jak opisaliśmy w raporcie 1:

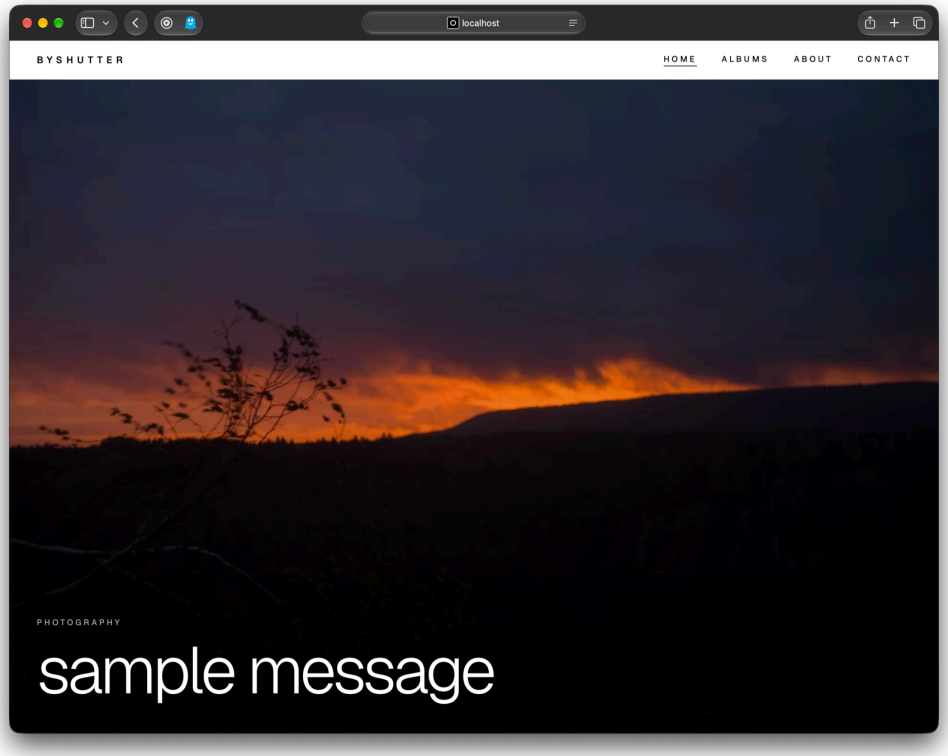
- zdjęcie tytułowe jest ustawiane stylem `background-image` dopiero po odpowiedzi API,
- profil strony głównej zwraca wyłącznie identyfikatory zdjęć, więc dla każdego zdjęcia klient wykonuje osobne żądanie o metadane (`GET /api/photos/{id}`), a dopiero potem pobiera plik z Caddy,

- zdjęcia kafelków na stronie głównej nie mają `loading="lazy"`, natomiast siatka albumu i okładki albumów mają,
- brak nagłówków cache i kompresji w Caddy (potwierdzone w pomiarze, rozdz. 3).

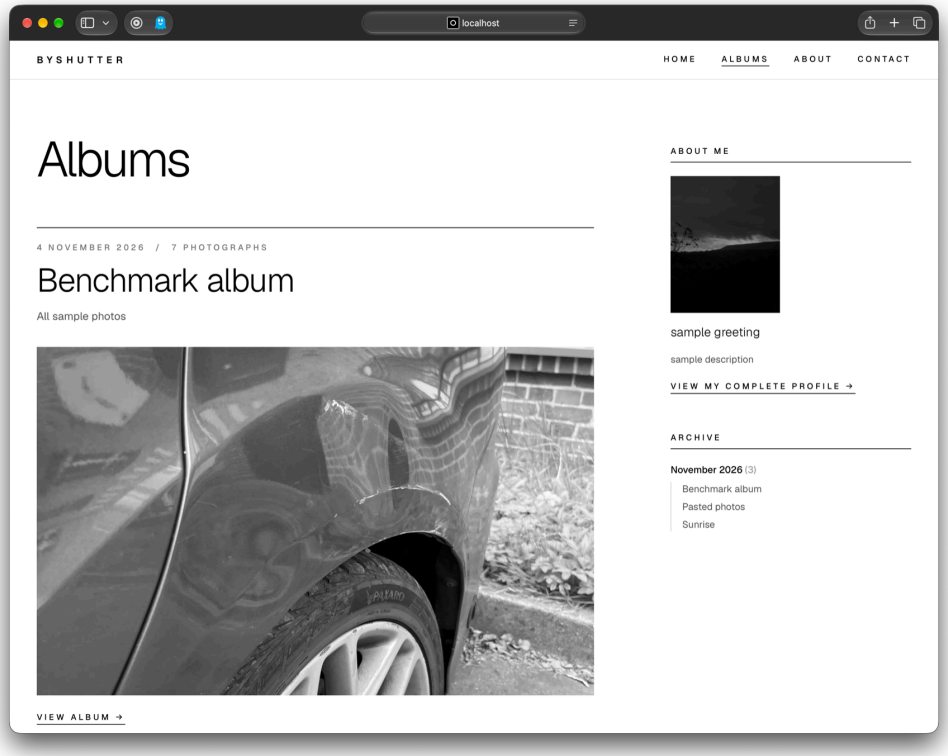
Różnice względem konfiguracji z raportu 1, które wpływają na porównywalność wartości bezwzględnych: czcionka (Geist, licencja OFL) jest hostowana lokalnie, a nie importowana z Google Fonts, więc pomiary nie zależą od sieci zewnętrznej; nie ma pliku zastępczego `ClockImage.png`; pakiet JavaScript jest mniejszy (ok. 94 kB, 36 kB po kompresji, wobec ok. 337 kB w raporcie 1). Pomiary wykonano na wersji produkcyjnej klienta zbudowanej przed dodaniem panelu administracyjnego (jego widoki są ładowane osobno i nie wpływają na strony publiczne), więc oba pomiary używają identycznego pakietu. Wartości bezwzględne metryk nie są więc wprost porównywalne z wartościami z etapu 1. Wniosek o wpływie zdjęć jest jednak przenaszalny, bo zdjęcia stanowią ponad 99 % przesyłanych bajtów (rozdz. 3), a klient niczego w nich nie zmienia.

2.4 Strony publiczne serwisu

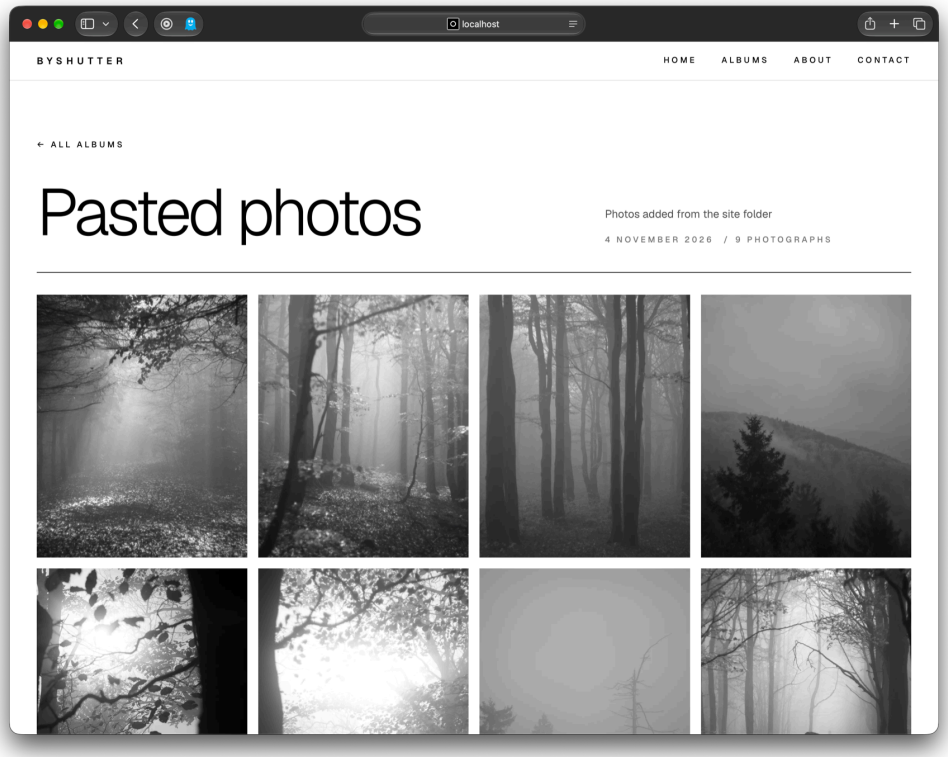
Poniższe zrzuty ekranu pokazują strony publiczne klienta, na których wykonano pomiary, w stanie po wgraniu danych testowych (rozdz. 2.2): stronę główną, listę albumów, widok albumu oraz podstrony „About” i „Contact”. Panel administracyjny (`/config`) nie jest częścią pomiarów i został pominięty.



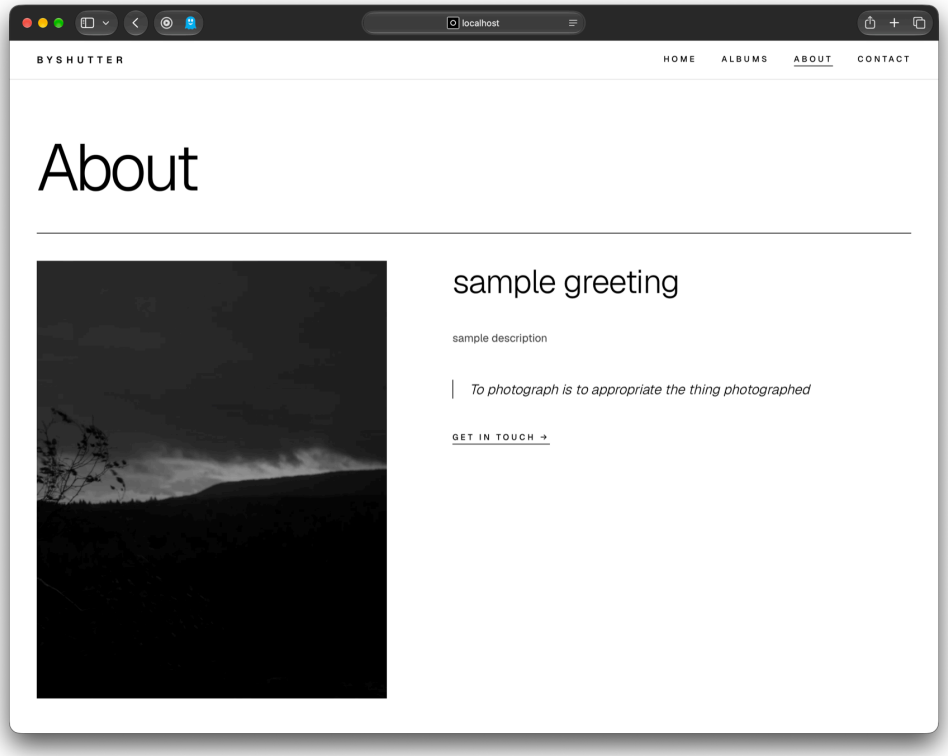
Rysunek 1: Strona główna (`/home`): zdjęcie tytułowe z hasłem i nawigacją



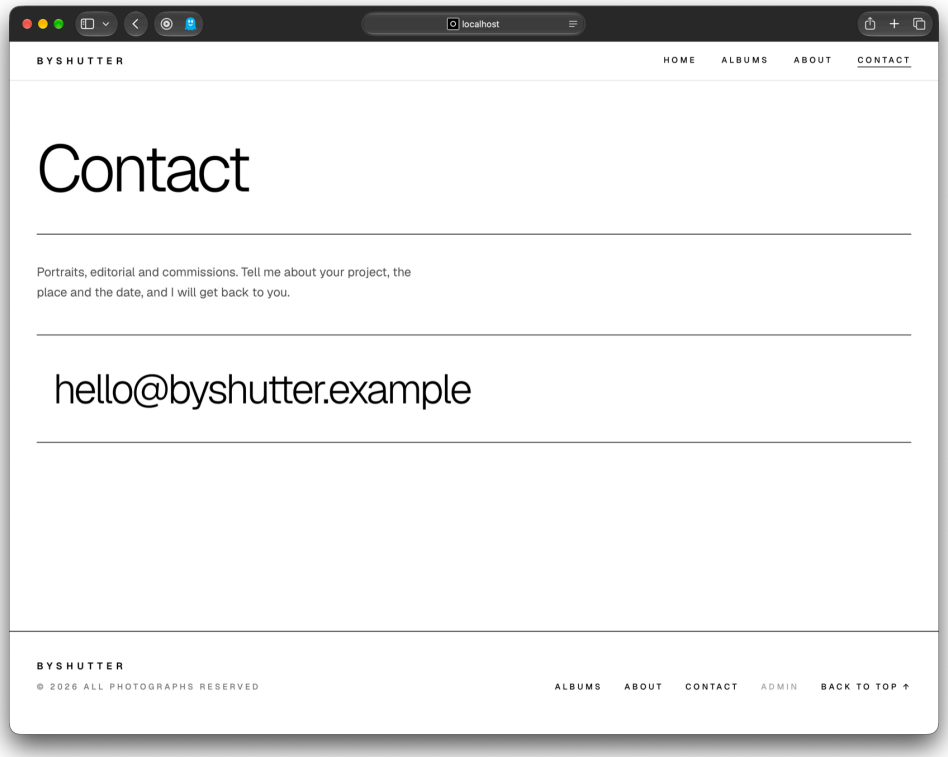
Rysunek 2: Lista albumów (`/albums`)



Rysunek 3: Widok albumu (`/albums/{id}`): siatka zdjęć



Rysunek 4: Podstrona „About” (`/about-me`)



Rysunek 5: Podstrona „Contact” (`/contact`)

2.5 Narzędzia, profile sieci i procedura

Lighthouse uruchamiamy programowo z *stosowanym* (nie symulowanym) ograniczaniem sieci i CPU, czyli przeglądarka rzeczywiście pobiera zasoby z zadaną przepustowością i opóźnieniem. Dzięki temu czasy są

obserwowane, jak w WebPageTest, a nie szacowane modelem. Profile odpowiadają nazwom używanym w WebPageTest, o których pisaliśmy w raporcie 1:

Profil	Urządzenie	Pobieranie	RTT	CPU
4G	emulacja telefonu (412 × 823 px, DPR 1,75)	9 Mbit/s	170 ms	4× wolniejszy
Cable	komputer stacjonarny (1350 × 940 px)	5 Mbit/s	28 ms	bez ograniczeń

Tabela 2: Profile sieci użyte w pomiarach

Każdą kombinację (strona główna i widok albumu, razy dwa profile) mierzymy pięciokrotnie z zimną pamięcią podręczną i podajemy medianę, zgodnie z założeniem z raportu 1. Dodatkowo, niezależnie od przeglądarki i bez ograniczania łącza, mierzymy programem `curl` rozmiar i czas pobrania każdego pliku (mediana z 10 powtórzeń) oraz czasy odpowiedzi API; to pokazuje wyłącznie koszt po stronie serwera.

2.6 Dlaczego bez WebPageTest

WebPageTest jest usługą internetową: testy uruchamiają się na zewnętrznych maszynach, które nie widzą adresu `localhost`. Uruchomienie testu wymagałoby wystawienia lokalnej aplikacji do internetu (z domyślnym kontem administratora) albo wdrożenia jej na publicznym serwerze, czego na tym etapie nie robiliśmy. Lighthouse z ograniczaniem łącza daje wszystkie pięć metryk czasowych z raportu 1 (TTFB, FCP, LCP, Load Time w postaci *Fully loaded*, CLS) oraz obie metryki ilościowe (rozmiar strony, liczba żądań), więc nie tracimy żadnej z zaplanowanych miar. Pomiar w WebPageTest warto powtórzyć po wdrożeniu serwisu na serwerze docelowym (p. rozdz. 8).

2.7 Uwaga o metryce Load Time

Zdarzenie `load` przeglądarki w aplikacji SPA następuje, zanim klient pobierze dane z API i zdjęcia: wartości zmierzone dla naszej strony to 0,25–0,7 s przy 24 MB zdjęć pobieranych jeszcze przez kilkadziesiąt sekund. Dlatego jako *Load Time* przyjmujemy *Fully loaded* (czas zakończenia ostatniego żądania sieciowego), czyli drugi wariant wskazany w definicji z raportu 1.

3 Stan wyjściowy

Pomiar wykonaliśmy na niezmienionym backendzie (zdjęcia przechowywane i wydawane w postaci wgranych plików) z pustą bazą i siedmioma dużymi zdjęciami z rozdz. 2.2.

Metryka	4G, strona główna	4G, album	Cable, strona główna	Cable, album
TTFB	2 ms	2 ms	2 ms	2 ms
FCP	505 ms	507 ms	184 ms	186 ms
LCP	21,8 s	20,8 s	36,9 s	42,8 s
Fully loaded	22,1 s	27,1 s	37,9 s	47,1 s
Load (zdarzenie `load`)	660 ms	657 ms	251 ms	249 ms
Speed Index	12,6 s	13,0 s	19,9 s	22,5 s
CLS	0,109	0,093	0,112	0,045
Total Blocking Time	0 ms	1,45 s	60 ms	2,96 s
Rozmiar strony (transfer)	24,16 MB	30,09 MB	24,16 MB	30,09 MB
w tym obrazy	24,07 MB	30,01 MB	24,07 MB	30,01 MB

Liczba żądań	21	15	21	15
Wynik Lighthouse (0–100)	62	37	62	35

Tabela 3: Stan wyjściowy: mediana z 5 przebiegów

3.1 Co pokazują pomiary

- **Zdjęcia to niemal cały transfer.** Strona główna pobiera 24,16 MB, z czego 24,07 MB (99,7 %) to sześć zdjęć. Pozostałe 21 żądań (osiem plików statycznych: 78 kB, siedem żądań do API: 4,6 kB) jest pomijalne pod względem bajtów. Album pobiera 30,0 MB zdjęć w siedmiu plikach.
- **LCP jest zdominowany przez pobieranie zdjęcia tytułowego.** Elementem LCP jest zdjęcie tytułowe ustawione jako tło CSS, jak przypuszczaliśmy w raporcie 1. Na profilu 4G z 21,8 s LCP aż 20,4 s (94 %) to czas pobierania obrazu; czas do pierwszego bajta (3 ms) i renderowanie (0,36 s) są marginalne. Na profilu Cable jest to 36,4 z 36,9 s. Zdjęcie tytułowe pobiera się tak długo, bo dzieli łącze z pięcioma pozostałymi dużymi zdjęciami ładowanymi równolegle (sumarycznie 24 MB przy 9 Mbit/s to ok. 21 s).
- **Adres zdjęcia tytułowego jest odkrywany późno.** Lighthouse potwierdza, że żądanie tego obrazu nie jest widoczne w dokumencie HTML i nie ma `fetchpriority=high`. Opóźnienie od startu do rozpoczęcia pobierania wynosi 1,0 s (4G) i 0,29 s (Cable): to koszt łańcucha żądań (profil, a potem metadane każdego zdjęcia). Jest to wartość drugorzędna wobec czasu pobierania, ale mierzalna.
- **FCP nie oddaje problemu.** Strona „rusza” po 0,18–0,51 s (tekst i układ), a zdjęcia dopływają przez 20–47 s. Dlatego do oceny wpływu zdjęć używamy LCP i *Fully loaded*.
- **Serwer jest szybki, kosztem są bajty.** Bez ograniczania łącza siedem zdjęć (30,0 MB) pobiera się z Caddy w łącznie 233 ms, a żądania API trwają 4–5 ms (TTFB ok. 2 ms).
- **Brak cache i kompresji.** Caddy wysyła tylko `ETag` i `Last-Modified`, bez `Cache-Control` (Lighthouse: czas życia w cache 0, szacowane 24 MB do zaoszczędzenia przy kolejnych wizytach) i bez kompresji. Kompresja nie pomogłaby plikom JPEG; odpowiedzi JSON z API są niewielkie.
- **Szacunek Lighthouse.** Audyt dostarczania obrazów szacuje, że z 24,07 MB da się zaoszczędzić ok. 22,0 MB (91 %).
- **Dodatkowo.** CLS na stronie głównej (0,109 i 0,112) jest nieznacznie powyżej progu 0,1, bo treść pojawia się po odpowiedzi API. Na stronie albumu Total Blocking Time wynosi 1,45 s (4G) i 2,96 s (Cable); prawdopodobną przyczyną jest dekodowanie przez przeglądarkę zdjęć o rozdzielczości 12–26 Mpix (wartość spada do zera po optymalizacji).

4 Optymalizacja nr 1: zmniejszenie zdjęć, WebP i oryginały

4.1 Wybór i uzasadnienie

Pierwszą optymalizacją jest pierwsza pozycja z listy zapowiedzianej w raporcie 1: zmniejszenie zdjęć i konwersja formatu. Wybraliśmy ją, ponieważ:

- zdjęcia to ponad 99 % transferu, a Lighthouse szacuje, że z 24,07 MB da się zaoszczędzić ok. 22,0 MB (91 %), więc jest to największa dźwignia,
- zmiana mieści się w tym repozytorium (backend) i nie wymaga zmian w kliencie: API zwraca teraz nazwę pliku `.webp`, a klient buduje adres tak samo jak wcześniej,
- zmienia jedną grupę zmiennych (rozmiar i format zdjęć), więc jej efekt można odizolować; pozostałe kandydatki (`srcset`, `loading="lazy"`, preload zdjęcia tytułowego, nagłówki cache, kompresja) zmieniają inne zmienne i będą mierzone osobno.

4.2 Co zmieniono

Przy wgrywaniu zdjęcia (`POST /api/photos`) serwer wykonuje teraz następujące kroki:

1. wczytuje plik do pamięci i dekoduje go biblioteką SixLabors.ImageSharp 3.1.12; plik, który nie jest obrazem, powoduje błąd *zanim* cokolwiek zostanie zapisane,

2. obraca piksele zgodnie z orientacją EXIF, a następnie zmniejsza zdjęcie tak, by dłuższy bok miał co najwyżej 2048 px (bez powiększania mniejszych zdjęć),
3. koduje wynik do WebP (z utratą, jakość 80); w wersji internetowej usuwane są metadane EXIF, IPTC i XMP (m.in. położenie GPS i numer seryjny aparatu), a profil kolorów ICC jest zachowany, żeby zdjęcia o szerokiej gamie nie traciły barw,
4. zapisuje wiersz metadanych: `FileName` to teraz nazwa pliku `.webp`, a nowa, opcjonalna kolumna `OriginalFileName` wskazuje oryginał (migracja `AddPhotoOriginalFileName`); API zwraca to pole w obiekcie zdjęcia,
5. zapisuje dwa pliki: nietknięty oryginał w `originals/<nazwa oryginału>` i wersję internetową `<nazwa>.webp`.

Kolejność kroków jest celowa: unikalny indeks na `FileName` odrzuca duplikat jeszcze przed zapisem plików, więc ponowne wgranie zdjęcia o tej samej nazwie nie nadpisze istniejącego pliku. Przy okazji zabezpieczyliśmy zapis: nazwa pliku z formularza jest sprowadzana do samej nazwy (`Path.GetFileName`), żeby nie dało się zapisać pliku poza katalogiem zdjęć.

Zmienione i dodane elementy: domena (`Photo.OriginalFileName`, interfejs `IImageOptimizer`), aplikacja (`OptimizeImageCommand`, zmienione `UploadImageCommand` i `CreatePhotoMetaCommand`, pole w `PhotoDto` i pięciu zapytaniach), infrastruktura (`ImageOptimizer`, tworzenie podkatalogu w `FileStorage`, konfiguracja EF i migracja) oraz warstwa Web (`PhotoEndpoints`, rejestracja usługi w `Program.cs`). Parametry (2048 px, jakość 80) są stałymi w klasie `ImageOptimizer`. Szerokość 2048 px wystarcza na zdjęcie tytułowe na ekranie Full HD i QHD; ekrany Retina dostaną w kolejnym etapie (`srcset`) większe warianty.

4.3 Oryginały i przycisk „View full image”

Zachowanie oryginału i przycisk pełnego zdjęcia nie są złym rozwiązaniem UX, tylko wzorcem spotykanym w serwisach fotograficznych, pod dwoma warunkami, które spełniliśmy: (1) oryginał jest pobierany *wyłącznie* po jawnym kliknięciu, nigdy w tle, więc nie wpływa na ładowanie strony; (2) przeglądarka zdjęć (lightbox) pokazuje wersję internetową, która jest już w pamięci podręcznej po wyświetleniu siatki. W kliencie przycisk pojawia się tylko wtedy, gdy API zwróci `originalFileName`, więc zdjęcia wgrane przed zmianą nadal działają.

Dwie uwagi do decyzji właściciela serwisu: oryginały leżą pod publicznym adresem `/originals/...` i zawierają komplet metadanych (w tym położenie GPS), a każdy może je pobrać w pełnej rozdzielczości. Jeśli fotograf nie chce udostępniać plików w pełnej rozdzielczości, przycisk można ukryć lub oryginały zabezpieczyć. Poza tym usuwanie zdjęcia w API usuwa dziś tylko wiersz w bazie, a nie pliki (także przed naszą zmianą), więc oryginały pozostają na dysku; do uzupełnienia przy porządkowaniu kodu.

4.4 Koszty i kompromisy

Optymalizacja przenosi koszt z odwiedzającego (przy każdym wyświetleniu) na administratora (jednorazowo, przy wgrywaniu): serwer dekoduje zdjęcie 12–26 Mpix i koduje je do WebP, więc wgrywanie trwa dłużej i zużywa więcej pamięci i CPU (wartości w rozdz. 5). Dodatkowo na dysku leżą dwa pliki na zdjęcie, ale wersja internetowa jest wielokrotnie mniejsza od oryginału, więc łączne zużycie dysku rośnie nieznacznie.

5 Pomiar po optymalizacji

Po wdrożeniu zmiany wyczyściliśmy bazę, usunęliśmy pliki testowe, zbudowaliśmy API od nowa, wgraliśmy te same siedem zdjęć tym samym skryptem i powtórzyliśmy identyczną procedurę pomiarową (ten sam klient, te same profile, 5 przebiegów). Rozrzut między przebiegami jest mały (np. LCP strony głównej na 4G: 3,73–3,76 s po zmianie i 21,78–21,80 s przed nią), więc różnice poniżej kilku procent traktujemy jako brak zmiany.

5.1 Metryki strony

Metryka	Przed	Po	Zmiana

TTFB	2 ms	2 ms	bez zmian
FCP	505 ms	510 ms	bez zmian
LCP	21,8 s	3,74 s	−83 %
Fully loaded (ostatnie żądanie sieciowe)	22,1 s	3,84 s	−83 %
Speed Index	12,6 s	3,03 s	−76 %
CLS	0,109	0,109	bez zmian
Rozmiar strony (transfer)	24,16 MB	3,14 MB	−87 %
w tym obrazy	24,07 MB	3,06 MB	−87 %
Liczba żądań	21	21	bez zmian
Wynik wydajności Lighthouse (0–100)	62	85	+23

Tabela 4: Strona główna, profil 4G (mediana z 5 przebiegów)

Metryka	Przed	Po	Zmiana
TTFB	2 ms	2 ms	bez zmian
FCP	184 ms	189 ms	+2,7 %
LCP	36,9 s	4,84 s	−87 %
Fully loaded (ostatnie żądanie sieciowe)	37,9 s	5,09 s	−87 %
Speed Index	19,9 s	3,62 s	−82 %
CLS	0,112	0,112	bez zmian
Rozmiar strony (transfer)	24,16 MB	3,14 MB	−87 %
w tym obrazy	24,07 MB	3,06 MB	−87 %
Liczba żądań	21	21	bez zmian
Wynik wydajności Lighthouse (0–100)	62	65	+3

Tabela 5: Strona główna, profil Cable (mediana z 5 przebiegów)

Metryka	Przed	Po	Zmiana
TTFB	2 ms	2 ms	bez zmian
FCP	507 ms	508 ms	bez zmian
LCP	20,8 s	2,28 s	−89 %
Fully loaded (ostatnie żądanie sieciowe)	27,1 s	4,39 s	−84 %
Speed Index	13,0 s	2,70 s	−79 %
CLS	0,093	0,093	bez zmian
Rozmiar strony (transfer)	30,09 MB	3,97 MB	−87 %
w tym obrazy	30,01 MB	3,89 MB	−87 %
Liczba żądań	15	15	bez zmian
Wynik wydajności Lighthouse (0–100)	37	96	+59

Tabela 6: Widok albumu, profil 4G (mediana z 5 przebiegów)

Metryka	Przed	Po	Zmiana

TTFB	2 ms	1 ms	bez zmian (poniżej 1 ms)
FCP	186 ms	188 ms	+1,1 %
LCP	42,8 s	2,90 s	−93 %
Fully loaded (ostatnie żądanie sieciowe)	47,1 s	6,37 s	−86 %
Speed Index	22,5 s	3,32 s	−85 %
CLS	0,045	0,045	bez zmian
Rozmiar strony (transfer)	30,09 MB	3,97 MB	−87 %
w tym obrazy	30,01 MB	3,89 MB	−87 %
Liczba żądań	15	15	bez zmian
Wynik wydajności Lighthouse (0–100)	35	76	+41

Tabela 7: Widok albumu, profil Cable (mediana z 5 przebiegów)

5.2 Składowe LCP strony głównej

Faza LCP	Przed	Po	Zmiana
Czas do pierwszego bajta (TTFB)	3 ms	3 ms	bez zmian
Opóźnienie rozpoczęcia pobierania obrazu	1,00 s	1,01 s	bez zmian
Czas pobierania obrazu	20,4 s	2,65 s	−87 %
Opóźnienie renderowania	358 ms	73 ms	−80 %

Tabela 8: Fazy LCP, strona główna, 4G

Faza LCP	Przed	Po	Zmiana
Czas do pierwszego bajta (TTFB)	3 ms	3 ms	bez zmian
Opóźnienie rozpoczęcia pobierania obrazu	285 ms	294 ms	+3,2 %
Czas pobierania obrazu	36,4 s	4,48 s	−88 %
Opóźnienie renderowania	157 ms	69 ms	−56 %

Tabela 9: Fazy LCP, strona główna, Cable

Cała poprawa pochodzi z fazy pobierania obrazu; opóźnienie rozpoczęcia pobierania (łańcuch żądań) nie zmieniło się. Elementem LCP strony głównej nadal jest zdjęcie tytułowe (tło CSS), a na stronie albumu pierwsze zdjęcie siatki.

5.3 Pliki zdjęć

Plik	Rozmiar przed	Rozmiar po	Zmiana	Czas pobrania przed	Czas pobrania po
`sample-01.webp`	4,43 MB	604 kB	−86 %	35 ms	8 ms
`sample-02.webp`	3,68 MB	469 kB	−87 %	29 ms	8 ms
`sample-03.webp`	4,21 MB	592 kB	−86 %	32 ms	8 ms
`sample-04.webp`	5,19 MB	810 kB	−84 %	40 ms	9 ms
`sample-05.webp`	3,34 MB	348 kB	−90 %	26 ms	7 ms

`sample-06.webp`	3,21 MB	230 kB	−93 %	26 ms	8 ms
`sample-07.webp`	5,94 MB	838 kB	−86 %	45 ms	10 ms
Razem	**30,00 MB**	**3,89 MB**	**−87 %**	**233 ms**	**58 ms**

Tabela 10: Rozmiar i czas pobrania plików przy łączu lokalnym bez ograniczeń (mediana z 10 powtórzeń)

Wersje internetowe mają od 230 do 838 kB (7–16 % rozmiaru oryginału). Typ pliku zmienił się z `image/jpeg` na `image/webp`; nagłówki cache pozostały bez zmian (brak `Cache-Control`). Odpowiedzi API trwają tyle samo (3–5 ms); pojedyncze obiekty zdjęć urosły o 35–253 B przez nowe pole `originalFileName`.

5.4 Koszt po stronie wgrywania i dysku

Plik	Rozmiar wgrywanego pliku	Czas wgrania przed	Czas wgrania po	Wydłużenie
`sample-01.jpeg`	4,43 MB	88 ms	3 823 ms	43×
`sample-02.jpeg`	3,68 MB	39 ms	2 233 ms	57×
`sample-03.jpeg`	4,21 MB	40 ms	2 313 ms	58×
`sample-04.jpeg`	5,19 MB	46 ms	2 489 ms	54×
`sample-05.jpeg`	3,34 MB	31 ms	1 918 ms	62×
`sample-06.jpeg`	3,21 MB	27 ms	1 772 ms	66×
`sample-07.jpg`	5,94 MB	48 ms	2 486 ms	52×
Razem	**30,00 MB**	**319 ms**	**17 034 ms**	**53×**

Tabela 11: Czas wgrania zdjęcia przez API (jedno wgranie na zdjęcie)

Wgranie zdjęcia trwa teraz 1,8–3,8 s zamiast ok. 40 ms (łącznie 17,0 s zamiast 0,3 s dla siedmiu zdjęć; pierwsze wgranie zawiera rozruch kodu, więc jest najwolniejsze). Jest to koszt jednorazowy, ponoszony przez administratora. Zajętość dysku wzrosła z 30,00 MB do 33,89 MB (+13 %), bo obok oryginałów (30,00 MB) leżą wersje internetowe (3,89 MB).

6 Wnioski i dalsze kroki

6.1 Wnioski

- **Optymalizacja usunęła główny problem.** Transfer strony głównej spadł z 24,16 do 3,14 MB (−87 %), a albumu z 30,09 do 3,97 MB (−87 %). LCP skrócił się o 83–93 % (strona główna: 21,8 do 3,74 s na 4G i 36,9 do 4,84 s na Cable; album: 20,8 do 2,28 s i 42,8 do 2,90 s), *Fully loaded* o 83–87 %, a Speed Index o 76–85 %. Wynik Lighthouse wzrósł z 62 do 85 (4G) i z 62 do 65 (Cable) na stronie głównej oraz z 37 do 96 i z 35 do 76 w albumie.
- **Total Blocking Time w albumie spadł do zera** (z 1,45 i 2,96 s), czyli przeglądarka nie dekoduje już obrazów o rozdzielczości 12–26 Mpix.
- **Niezmienione** pozostały TTFB (ok. 2 ms), FCP (różnice poniżej 3 %), liczba żądań i CLS, co jest zgodne z oczekiwaniem: zmiana dotyczy tylko rozmiaru i formatu zdjęć.
- **Wpływ jest największy tam, gdzie jest najwięcej bajtów i najwolniejsze łącze**, a w absolutnych liczbach największy na profilu Cable (LCP albumu: −39,9 s).
- **Cena:** wgrywanie jest ok. 50 razy wolniejsze (1,8–3,8 s na zdjęcie, jednorazowo), dysk zajmuje o 13 % więcej, a jakość zdjęć na stronie jest nieco niższa niż oryginału (przycisk „View full image” daje dostęp do oryginału).

6.2 Czego optymalizacja nie rozwiązała

Strona główna nadal przekracza próg „dobrego” LCP (2,5 s): 3,74 s na 4G i 4,84 s na Cable. Faza pobierania obrazu trwa 2,65 s, czyli tyle, ile pobranie wszystkich sześciu zdjęć (3,06 MB przy 9 Mbit/s): zdjęcie tytułowe (604 kB) kończy się dopiero, gdy łącze obsłuży także pięć pozostałych zdjęć. Lighthouse nadal szacuje ok. 2,5 MB możliwych oszczędności na obrazach, głównie dlatego, że kafelki i zdjęcie profilowe są wyświetlane w rozmiarze rzędu 300 px, a pobierane w 2048 px.

6.3 Kolejne optymalizacje (etap 3)

1. **Warianty rozmiarów i `srcset`.** Generować przy wgrywaniu mniejsze wersje (np. 480, 960 i 2048 px) i używać `srcset`/`sizes` dla kafelków, okładek, siatki albumu i zdjęcia profilowego; to ma największy potencjał (kafelki nie muszą pobierać 2048 px). Szacujemy, że transfer strony głównej spadłby wówczas poniżej 1,5 MB (do zweryfikowania pomiarem).
2. **Zdjęcie tytułowe jako `<img>` z `fetchpriority="high"` i `preload`**, żeby przeglądarka odkrywała je wcześnie i uprzywilejowała w kolejce.
3. **`loading="lazy"` dla zdjęć poza ekranem** (kafelki i zdjęcie profilowe na stronie głównej), żeby nie konkurowały z LCP.
4. **Nagłówki cache** (`Cache-Control: public, max-age=31536000, immutable` dla zdjęć, których nazwy są unikalne), co wyeliminuje ponowne pobieranie przy kolejnych wizytach.
5. **Nazwa pliku w odpowiedzi profilu strony głównej**, co usunie sześć żądań o metadane zdjęć i skróci łańcuch (0,3–1,0 s opóźnienia przed LCP).
6. **Stabilizacja układu** (stałe proporcje elementów zależnych od danych z API), żeby CLS spadł poniżej 0,1.

7 Ograniczenia badania

- **Sieć lokalna.** Wszystko działa na jednej maszynie pod adresem `localhost`; czasy po stronie serwera (TTFB ok. 2 ms) nie odpowiadają połączeniu przez internet. Ograniczanie łącza realizuje przeglądarka (protokół DevTools), a nie zewnętrzne kształtowanie ruchu jak w WebPageTest.
- **Konfiguracja klienta.** Wartości bezwzględne zależą od konfiguracji klienta (rozdz. 2.3: czcionka lokalna, mniejszy pakiet, brak pliku zastępczego) i nie są wprost porównywalne z wynikami etapu 1; porównanie przed i po optymalizacji w tym raporcie jest jednak jednoznaczne, bo dotyczy tego samego pakietu.
- **Jedna maszyna, jeden zestaw zdjęć.** Stopień kompresji zależy od treści zdjęć (tu głównie sceny miejskie ze zdjęciami z telefonu i jednym zdjęciem z Pexels). Przebiegi są bardzo powtarzalne (rozrzut pomiędzy przebiegami podajemy w danych w `bench/results`), ale to nadal jedno środowisko sprzętowe.
- **Jakość zdjęć.** Jakość WebP 80 oceniliśmy wzrokowo, bez miary obiektywnej (np. SSIM); wybór parametrów jest kompromisem, który warto potwierdzić z fotografem.
- **Nie mierzyliśmy ponownej wizyty.** Optymalizacja nie zmienia nagłówków cache, więc pomiary dotyczą wejścia z pustą pamięcią podręczną.
- **WebPageTest.** Nie został użyty (rozdz. 2.5).
- **Biblioteka i jej licencja.** Wersje SixLabors.ImageSharp 3.2 i 4.x wymagają klucza licencyjnego już przy kompilacji, więc użyliśmy 3.1.12 (ostatnia wersja bez tego wymogu, która nie dostanie już poprawek). Licencja Six Labors (Split License) jest bezpłatna m.in. dla projektów open source, a w części zastosowań komercyjnych wymaga płatnej licencji. Przed wdrożeniem komercyjnym należy sprawdzić warunki albo użyć biblioteki na licencji MIT (np. SkiaSharp, NetVips); interfejs `IImageOptimizer` ogranicza taką zmianę do jednej klasy.

8 Powtarzalność pomiarów

Wszystkie skrypty są w katalogu `bench/` (opis w `bench/README.md`), a surowe wyniki obu pomiarów w `bench/results/baseline.json` i `bench/results/optimised.json`. Procedura:

1. `bench/reset.sh` tworzy kontenery od nowa (pusta baza), usuwa wygenerowane pliki testowe i buduje API,

2. `cd ByShutterPage; npm ci; npm run build; npx vite preview` uruchamia klienta na porcie 5173,
3. `node bench/seed.mjs` wgrywa siedem zdjęć i tworzy treść,
4. `CHROME_PATH=... node bench/measure.mjs <etykieta>` wykonuje pomiar Lighthouse i `curl`,
5. `node bench/summarize.mjs baseline optimised` zestawia wyniki.

Aby wykonać pomiar WebPageTest, należy wdrożyć serwis pod publicznym adresem (HTTPS z zaufanym certyfikatem), uruchomić test dla adresu strony głównej i widoku albumu na profilach 4G i Cable, z co najmniej pięcioma powtórzeniami, i porównać *Start Render*, *LCP*, *Fully Loaded* oraz *Bytes In* z wartościami z rozdz. 3 i 5.

9 Źródła

- web.dev – Time to First Byte, First Contentful Paint, Largest Contentful Paint, Cumulative Layout Shift: web.dev/articles/ttfb, fcp, lcp, cls.
- Dokumentacja Lighthouse: developer.chrome.com/docs/lighthouse.
- Dokumentacja WebPageTest: docs.webpagetest.org.
- Format WebP: developers.google.com/speed/webp.
- SixLabors.ImageSharp: docs.sixlabors.com.
- Raport 1: *Badanie wydajności serwisu ByShutter – raport, etap 1* (Docs/raport-1.pdf).
- Repozytorium projektu: ByShutter-FrontPage (backend `ByShutterBackend`, klient `ByShutterPage`, skrypty pomiarowe `bench`).